```
    var converter = new CurrencyConverter();                ❸
    decimal value = 3;                                      ❹
    decimal rate = 1.5m;                                    ❹
    int dp = 4;                                             ❹
    decimal expected = 2;                                   ❺

    var actual = converter.ConvertToGbp(value, rate, dp);   ❻

    Assert.Equal(expected, actual);                         ❼
}
```

❶ The [Fact] attribute marks the method as a test method.

❷ You can call the test anything you like.

❸ The class to test, commonly called the “system under test”

❹ The parameters of the test that will be passed to ConvertToGbp

❺ The result you expect

❻ Executes the method and captures the result

❼ Verifies that the expected and actual values match; if they don’t, throws an exception

This is your first proper unit test, which has been configured using Arrange, Act, Assert (AAA) style:

- *Arrange*—Define all the parameters and create an instance of the system (class) under test (SUT).
- *Act*—Execute the method being tested, and capture the result.
- *Assert*—Verify that the result of the Act stage had the expected value.

Most of the code in this test is standard C#, but if you’re new to testing, the `Assert` call will be unfamiliar. This is a helper class provided by xUnit for making assertions about your code. If the parameters provided to `Assert.Equal()` aren’t equal, the `Equal()` call will throw an exception and fail the test. If you change the `expected` variable in listing 35.4 to `2.5` instead of `2`, for example, and run the test, Test Explorer shows a failure, as you see in figure 35.6.